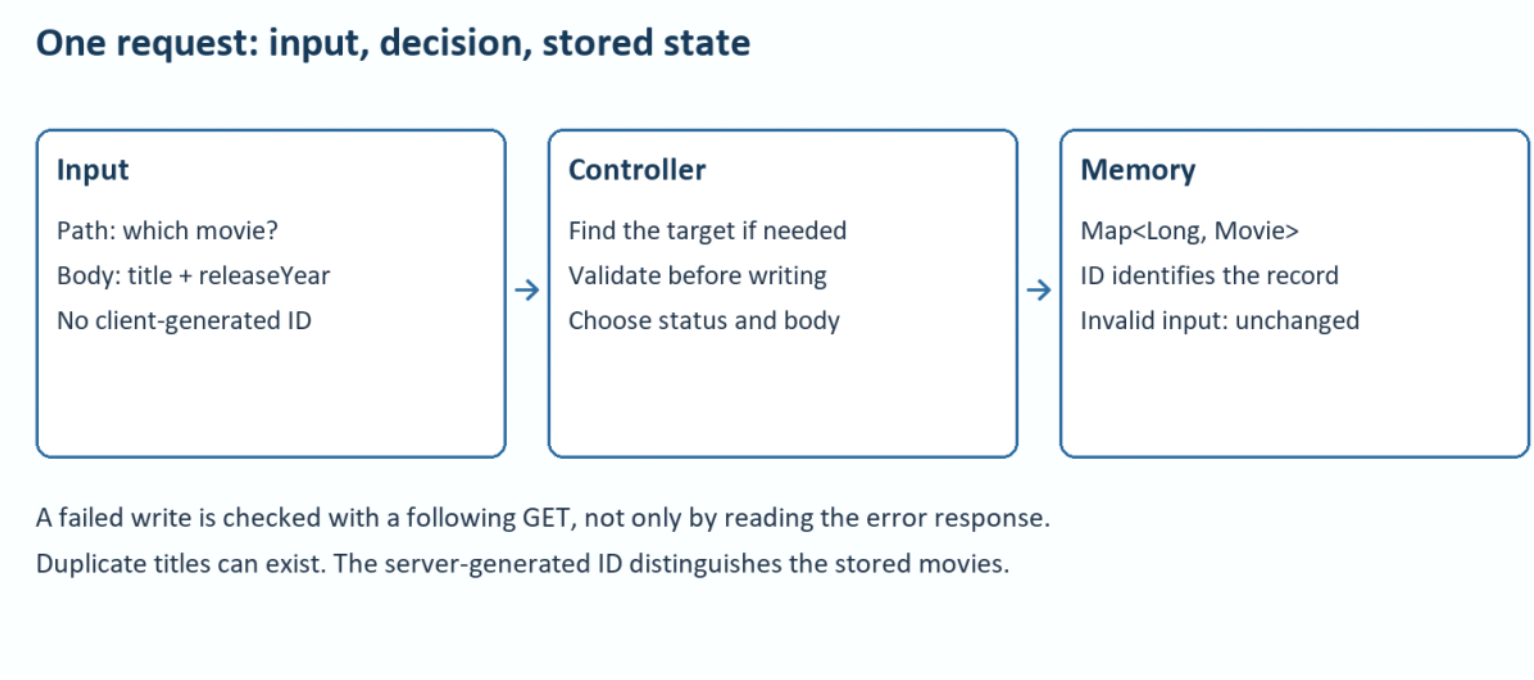
Lecture 8 - Online Watchlist revision and practice

Friday, October 23, 2026 10:06 AM

Lecture 8 — Online revision and Watchlist homework

Revision of Lecture 7 using the assigned Watchlist API, followed by a separate in-class question.

1. Predict the request, response and state




```
PUT /movies/2
{"title":"Interstellar - Revised","releaseYear":2015}
```

Which value identifies the stored record? Which values replace its details? Which fields belong in MovieInput and Movie?

How should GET /movies respond when empty? How should GET /movies/999 respond when that ID is absent?

```
PUT /movies/2
{"title":"Changed title","releaseYear":1800}
GET /movies/2
```

What should the second request prove about the rejected first request?

2. Assigned homework — Independent Watchlist CRUD API

Independent Watchlist CRUD API

Movie fields: id, title, releaseYear. Server assigns the ID. Store movies in memory.

```
GET /movies
GET /movies/{id}
POST /movies
PUT /movies/{id}
DELETE /movies/{id}
```

Create/update JSON:
{"title":"Interstellar","releaseYear":2014}

Rules: required nonblank title; releaseYear required, 1888–2100 inclusive. PUT replaces existing details and retains the ID. Invalid writes must leave existing data unchanged. Duplicate titles are allowed.

Use 201 plus Location for creation; 204 without a body for successful deletion; 404 for missing IDs; 400 for invalid input. Do not allocate IDs with collection size + 1.

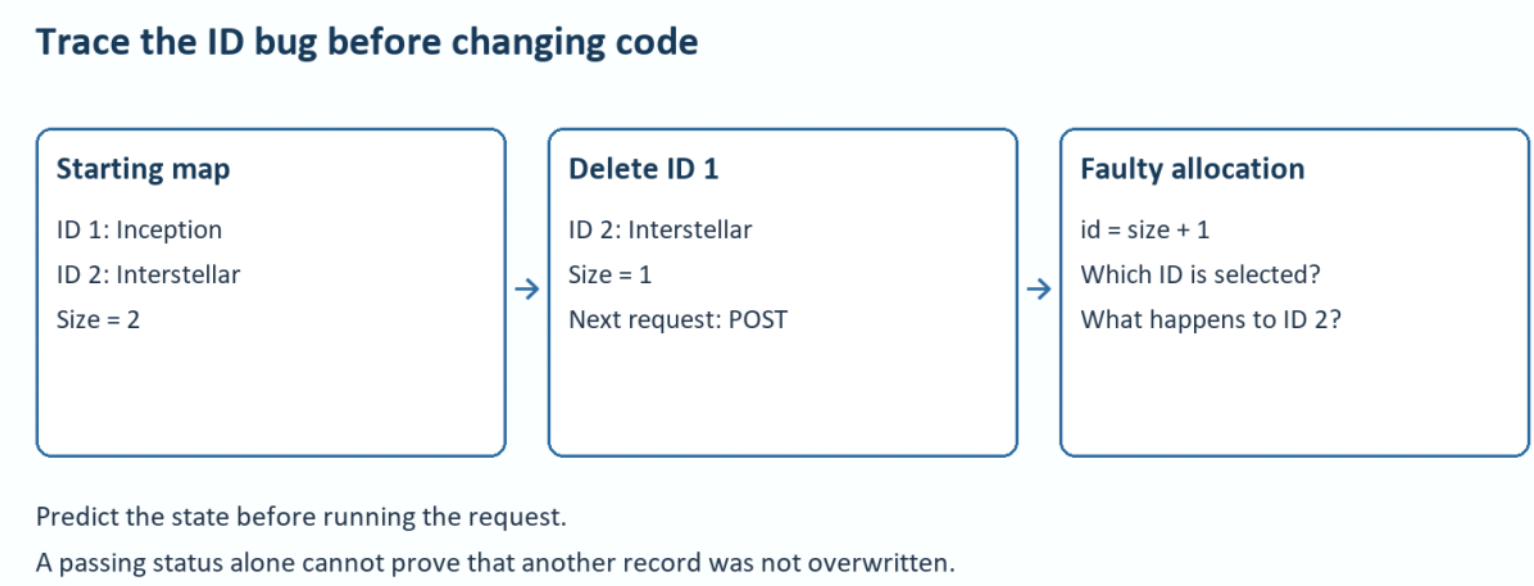
Submit source and request evidence: create/read/update/delete; rejected update followed by unchanged GET; delete an early ID then create without overwriting another movie; empty collection; repeated POST versus PUT.

Inspect the models

```
Movie:      long id, String title, int releaseYear
MovieInput: String title, Integer releaseYear
```

Why can the input year be null? When should trimming, unboxing, ID allocation and map writes happen?

ID trace




```
long id = movies.size() + 1; // Diagnose this allocation rule.
```

A rejected update

```
// Target exists. Diagnose this ordering:
movies.put(id, new Movie(id, input.getTitle(), input.getReleaseYear()));
String problem = validate(input);
if (problem != null) {
    return ResponseEntity.badRequest().body(new ApiError(problem));
}
```

What would GET return after a numeric but invalid year was rejected? Which operations must move before the write?

Submission evidence to demonstrate

Create/read/update/delete; rejected update followed by unchanged GET; delete an early ID and create without overwriting another movie; empty collection; repeated POST versus PUT. Duplicate titles remain allowed.

3. Separate in-class question — Movie Night Picks

This is a follow-on exercise, not part of the submitted Watchlist homework.
Continue the in-memory Watchlist. No new Spring feature is needed.

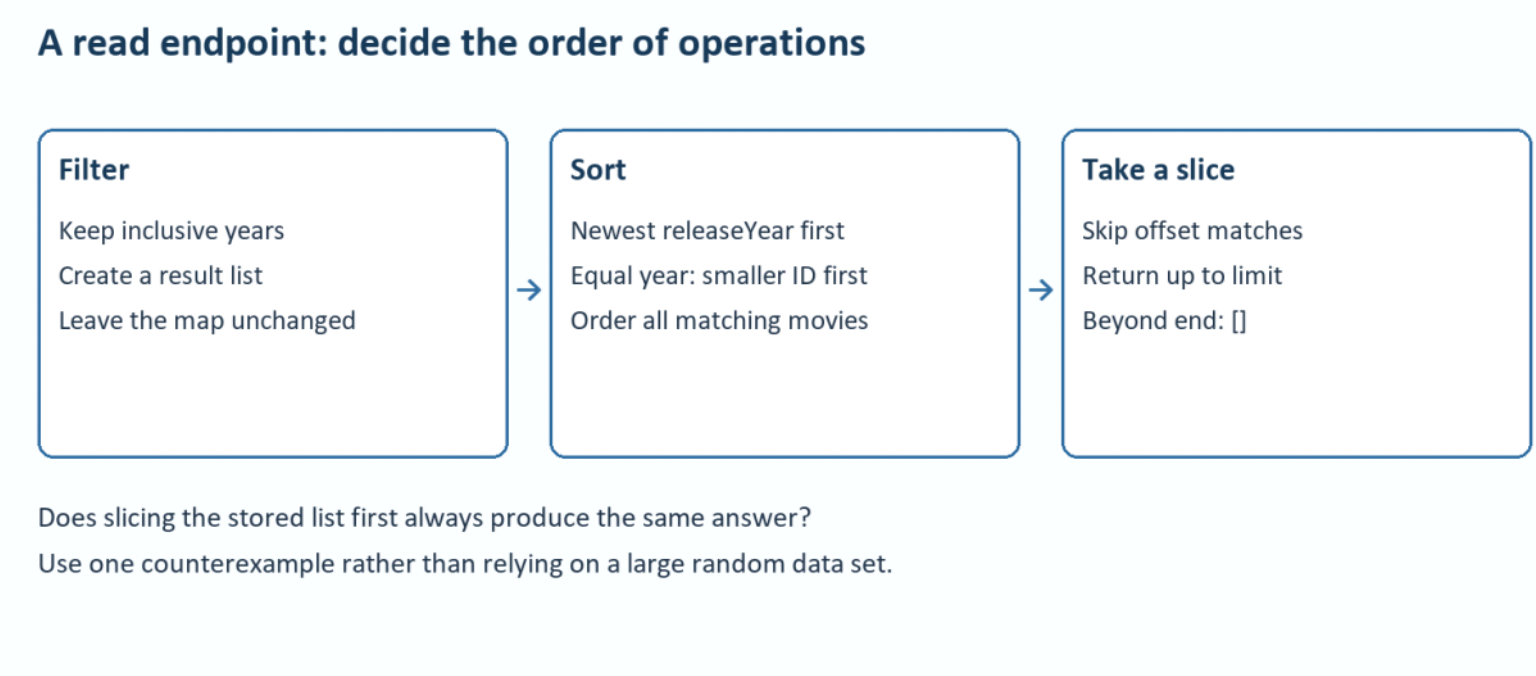
Add GET /movies/night-picks?minYear=2014&maxYear=2020&offset=1&limit=2

- minYear and maxYear are required, inclusive and within 1888–2100; minYear <= maxYear.
- offset defaults to 0 and cannot be negative. limit defaults to 3 and must be 1–20.

- Select only movies in the year range.
- Sort by releaseYear descending; if years match, by id ascending.
- Apply offset and limit to the filtered, sorted results. Return a JSON array.
- No matches or an offset beyond the results returns 200 with []. Invalid values return 400.
- The request must not change stored movies. No wrapper/page model is needed.

Shared example data (IDs are examples; use actual returned IDs in a running app):
1 Inception 2010
2 Interstellar 2014
3 Arrival 2016
4 Dunkirk 2017
5 Tenet 2020
6 Soul 2020

Before coding, predict the selected IDs, their sorted order and the final two IDs
for the example request. Then implement and verify boundary years, an empty match,
an oversized offset and unchanged GET /movies before/after the request.



Predict the matching IDs, their sorted order and the returned slice before writing code. Which operation must happen first? What happens when there are no matches?

Implementation checklist

```
1. Bind and validate query parameters
2. Build a matching list without changing stored data
3. Sort: year descending, then ID ascending
4. Take offset/limit from those ordered matches
5. Return the list or a validation error
```

Verify both year boundaries, equal years, an empty result, a large offset and unchanged GET /movies before/after the read.

4. Design a counterexample

A. An implementation slices all stored movies first, then filters/sorts. Find a request that proves it is wrong.

```
B. movie.getReleaseYear() > minYear && movie.getReleaseYear() < maxYear
C. int start = offset * limit;
```

For B and C, choose a request with an exact expected result. Explain why a “normal happy-path” request might miss the defect.

5. Exit checks

Give one request pair proving a rejected update preserved data; one ID-collision trace; and one test that checks operation order in Movie Night Picks.